

Input-Output

Tipi di istruzioni a livello ISA

- I/O programmato con busy waiting
- I/O controllato dall'interrupt
- I/O in DMA (Direct Memory Access)

Comunicazione tra CPU e moduli di I/O

La CPU comunica con i device di I/O tramite i **controllori**, che hanno il compito di trasformare:

- i comandi della CPU in segnali elettrici per le periferiche
- i segnali delle periferiche in dati per la CPU

Ogni controllore ha al suo interno dei **registri** identificati da un indirizzo, che può:

- **appartenere allo stesso insieme di indirizzi di memoria (memory mapped I/O)**
- **essere un indirizzo dedicato (isolated I/O)**

La comunicazione con i controllori è simile agli accessi in memoria

Registri nei controllori

- *dati* (di ingresso e uscita)

Es., per una stampante i dati da stampare

- *comandi* (dalla CPU alla periferica)

Es., i comandi di stampa

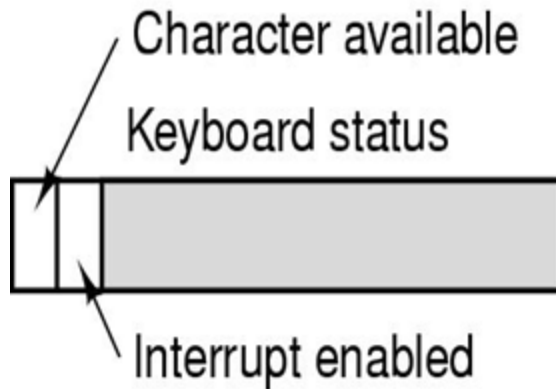
- *informazioni sullo stato della periferica*

(dalla periferica alla CPU)

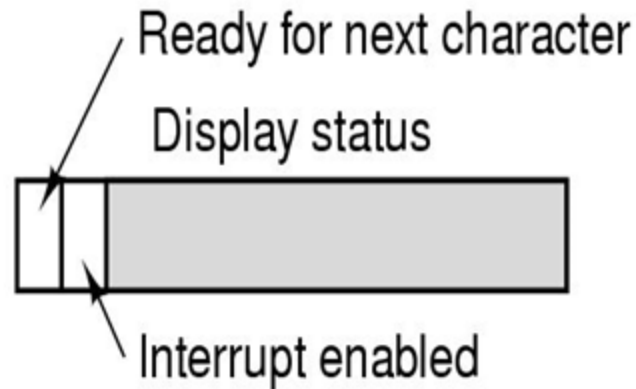
Es. Informazione sullo stato della stampante o sull'avanzamento dei lavori.

I/O programmato con busy waiting

La CPU controlla lo stato di avanzamento del I/O e lo stato della periferica *ispezionando in un ciclo il bit del registro* di stato del controllore READY fino a che questi non segnala di essere pronto per un nuovo comando di I/O



Keyboard buffer
Character received



Display buffer
Character to display

Svantaggio

la CPU rimane in ciclo attendendo che il dispositivo sia pronto, *busy waiting*

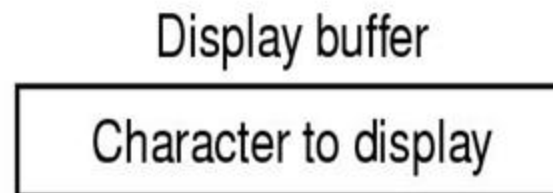
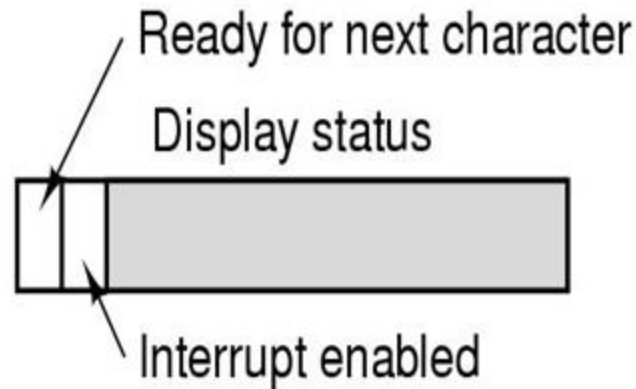
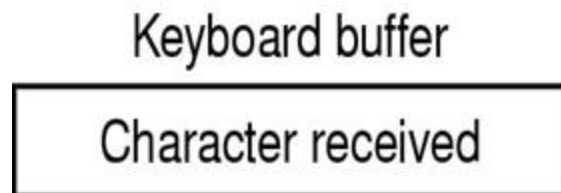
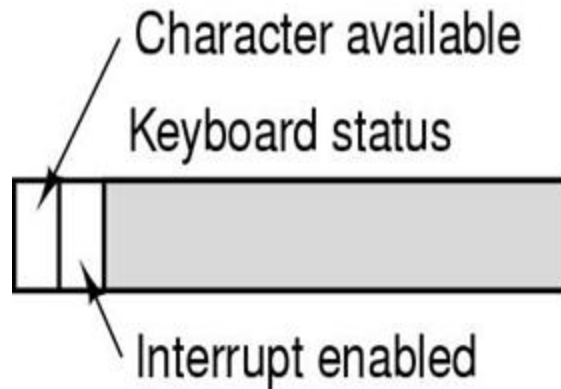
Polling

verifica ciclica (periodica) dei dispositivo di I/O tramite i bit di controllo

I/O programmato con busy waiting

Lettura es. keyboard:

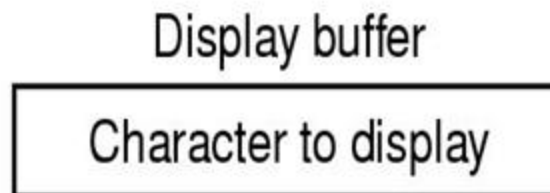
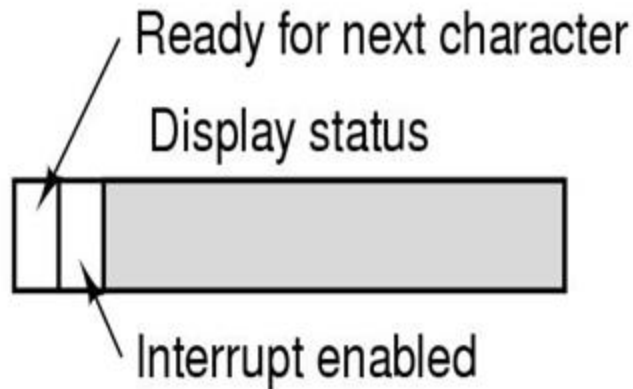
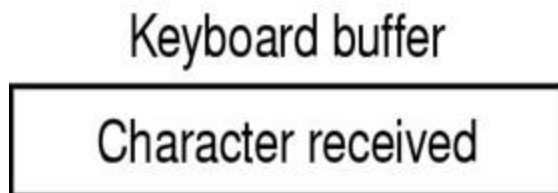
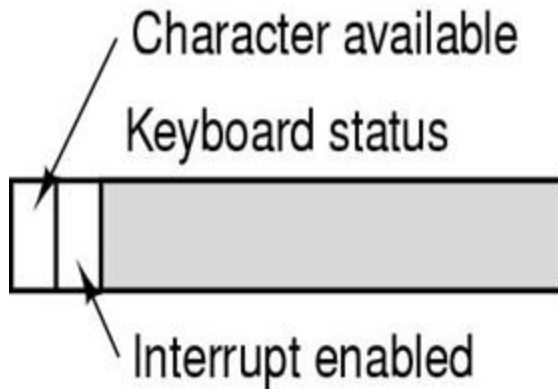
- Il **controllore** della tastiera alla pressione di un tasto mette a 1 un particolare bit del registro di stato e inserisce il codice ASCII del carattere associato al tasto nel registro buffer
- La **CPU** legge ripetutamente il bit del registro di stato fino a quando lo trova uguale a 1
- La **CPU** legge il registro buffer, trasferisce il valore per l'elaborazione e azzerava il bit del registro di stato



I/O programmato con busy waiting

Scrittura es. display:

- la **CPU** attende che il dispositivo sia pronto ad accogliere un carattere (bit Ready = 1)
- la **CPU** invia il carattere sul display buffer
- il **controllore** azzerava il bit Ready e visualizza il contenuto del buffer
- il **controllore** segnala di essere pronta a ricevere un nuovo comando (bit Ready = 1)



Interrupt

- Permette di liberarsi dal busy waiting:

il *dispositivo* quando ha finito *genera un segnale* che avverte la CPU di aver completato il proprio lavoro (*interrupt*)
- La CPU può abilitare l'interrupt mettendo a 1 un opportuno bit

Interrupt hardware

- Gli interrupt hardware sono cambiamenti nel flusso di controllo legati alla gestione dei periferici e non al programma stesso
- Gli interrupt vanno sempre dai dispositivi alla CPU
- Il dispositivo che alza un'interruzione asserisce una linea del bus, che è direttamente connessa alla CPU (per esempio a uno o più bit di un registro)
- L'interrupt interrompe il programma in atto e trasferisce il controllo ad un gestore di interrupt che eseguirà le azioni appropriate
- Una volta terminata la gestione dell'interrupt il controllo torna al programma interrotto

Interrupt

- Esempio: scrittura sullo schermo di una riga di “count” caratteri puntata da “ptr”, gestita con interruzione a livello del singolo carattere (il controller del dispositivo è in grado di visualizzare un solo carattere alla volta)
- **AZIONI HW:**
 - Il controller attiva una linea di interrupt del bus di sistema
 - Quando la CPU è pronta a gestire l'interruzione manda un ack sul bus
 - Il controller invia un intero sulle linee dati (vettore di interrupt)
 - La CPU preleva il vettore di interrupt dal bus e lo salva
 - La CPU impila PC ed altri registry di stato sullo stack
 - Il vettore di interrupt è usato come indice di memoria per trovare il valore di PC a cui si trova il gestore di quel tipo di interruzione
 - Si modifica PC, e a volte si evita che un'altra interruzione possa interrompere la gestione dell'interruzione in corso di esecuzione

Interrupt

AZIONI SW:

- Il codice ISA di gestione delle interruzioni salva tutti i registri (stato del programma che è stato interrotto)
- Si possono leggere tutte le informazioni dal buffer del dispositivo
- Se c'è stato un errore di I/O lo si gestisce
- Le variabili **ptr** e **count** vengono aggiornate, se ci sono altri caratteri da visualizzare si copia il carattere puntato da **ptr** nel buffer del controller del dispositivo
- Eventuali altri ack di fine trattamento interruzione
- Ripristino registri
- Esecuzione di una apposita istruzione di “return from interrupt – RETI” che ripristina modalità e stato della CPU.

Interrupt

- Gli interrupt sono **asincroni** rispetto al programma
- Gli interrupt devono essere gestiti in modo **trasparente**: lo stato dell'esecuzione dopo la gestione dell'interrupt deve tornare esattamente come era prima dell'interrupt stesso

Interrupt

- E se ci sono *più dispositivi* che lavorano su interruzione?
 - Il gestore delle interruzioni **disabilita** le interruzioni
 - Oppure, si definiscono delle **priorità** tra i dispositivi che possono generare delle interrupt e si permette che una sopravvenuta interruzione interrompa la gestione dell'interrupt precedente
- La priorità è proporzionale alla criticità del dispositivo che solleva l'interrupt
- Ad ogni dispositivo è assegnato un livello di priorità e un tipo di interruzione (**mascherabile o non mascherabile**)
- Se la CPU non permette più livelli di priorità si utilizza normalmente un chip apposito che gestisce in modo “autonomo” le interruzioni

Trap (o eccezione o interrupt sincroni)

- Sono chiamate a procedure automatiche che possono essere causata dal verificarsi di qualche **condizione eccezionale** oppure da istruzioni apposite per richiedere **servizi di base del sistema operativo**
- **Gestore delle trap**: procedura di gestione della richiesta di trap la cui posizione (indirizzo) è memorizzata in una locazione fissa a cui il programma salta in caso di trap
- L'eventuale eccezione è individuata dall'hardware
- Es. gestione dell'overflow (il test è fatto via hardware) in parallelo con l'esecuzione di altre funzionalità
- Le trap sono **sincrone** al programma

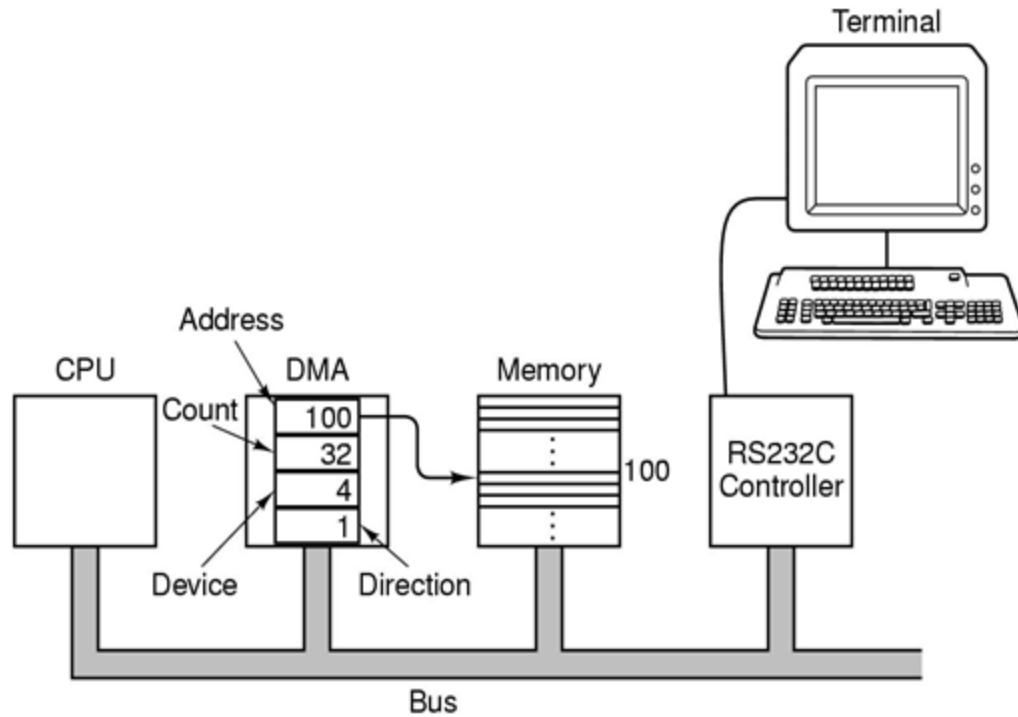
Trap (o eccezione)

Esempio, due modi diversi di gestione dell'overflow:

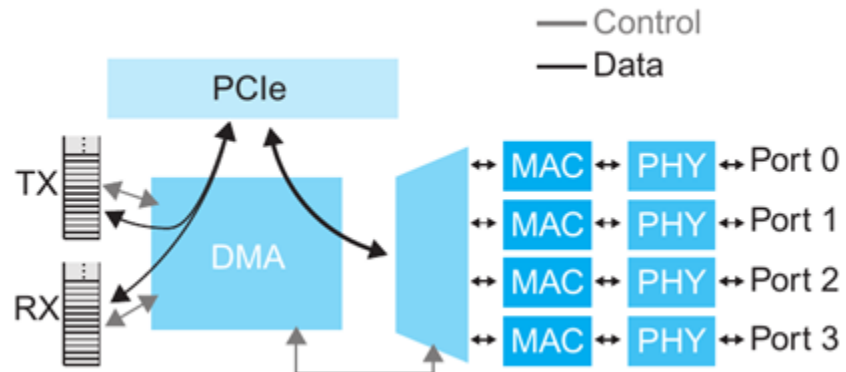
- *Senza trap*: l'hw setta un bit in un apposito registro e il programmatore ISA, se lo desidera, dopo un'istruzione che può causare overflow, testa il registro e salta ad apposita procedura
- *Con trap*: la condizione di overflow (rilevata da hw) genera una trap, che modifica il flusso di controllo ad una locazione prefissata (gestore della trap)

Differenze: nel primo caso ci sono molte più istruzioni ISA per il controllo dell'overflow sparse nel programma, nel secondo caso c'è una modifica dell'unità di controllo che alla fine dell'esecuzione delle istruzioni aritmetiche, controlla l'overflow

DMA (Direct Memory Access)



- I/O programmato ma svolto da un'apposita componente con accesso diretto al bus
- Il DMA è impostato direttamente dal software o il sistema operativo inizializzando opportuni registri
- La CPU e il DMA si contendono l'uso del bus



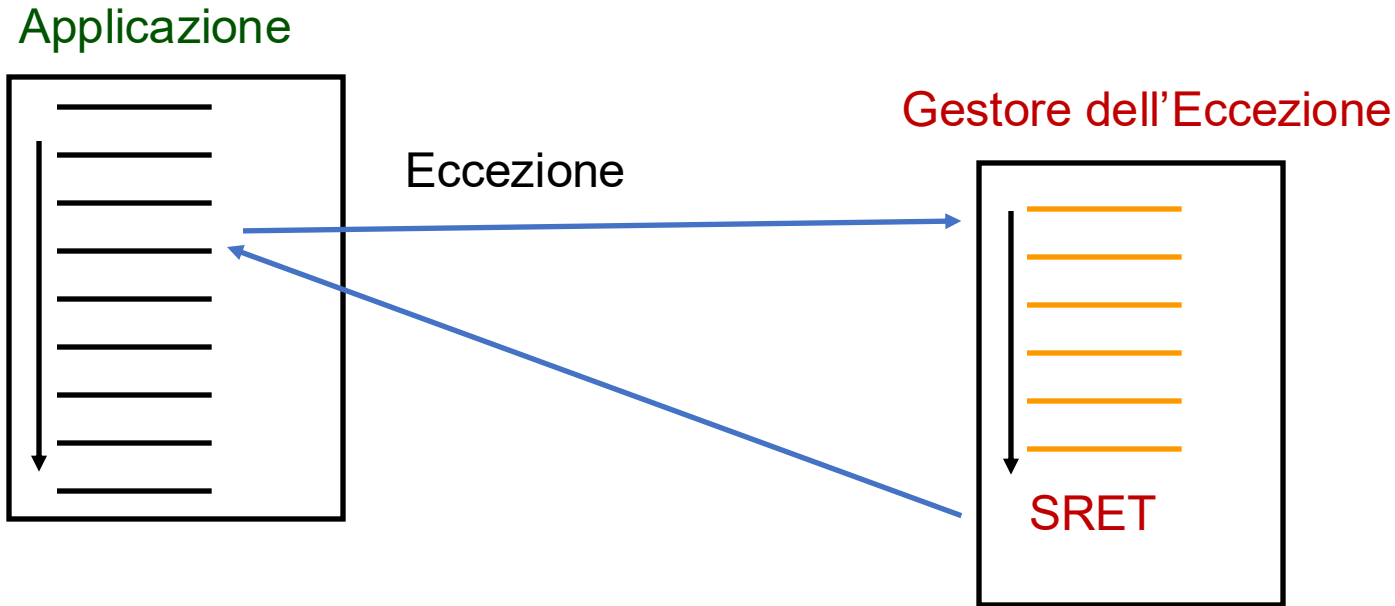
Il meccanismo di interruzione del processore RISC-V

Materiale aggiuntivo

slides da: <http://www.dii.unisi.it/~giorgi/didattica/arc1>

Eccezioni

SRET=Supervisor Return
è una istruzione speciale
a conclusione del gestore



- **Eccezione**

- trasferimento del flusso di controllo non legato strettamente al codice utente
- Il sistema gestisce l'eccezione eseguendo il "**Gestore dell'eccezione**"

- **Esempio**

- l'utente preme un tasto e questo interrompe l'esecuzione del programma utente
- la CPU esegue un "gestore della pressione di un tasto" che legge un codice corrispondente al tasto e lo trasferisce in un buffer relativo ai tasti premuti

Comportamento del calcolatore

- Il programma in esecuzione deve essere **sospeso** e poi **riattivato**
 - il calcolatore salva il punto del codice (Program Counter) in cui si è verificata l'eccezione (1)
- Il controllo passa quindi al “Gestore dell'Eccezione” (**Exception Handler**)
 - Il gestore dell'eccezione deve rimediare alla situazione
 - A seconda del tipo di eccezione, il gestore esegue azioni diverse
 - il calcolatore identifica e salva la causa dell'eccezione (2)
- **RISC-V: registri speciali SEPC e SCAUSE**
 - (1) **SEPC** - l'indirizzo del codice in cui si verifica l'eccezione
 - (2) **SCAUSE** - la causa dell'eccezione

Tipo di Eccezioni

Evento Asincrono

Interrupt
(e.g., I/O)

Errore

(e.g., overflow)

Evento Sincrono

Environment Call

(o "Trap" o "System Call")

Environment break

(debug)

L'eccezione permette alle applicazioni di avere accesso al calcolatore in modo controllato, attraverso codice che sta nel sistema operativo.



- **Modalità supervisor (kernel mode):**
durante l'eccezione il calcolatore esegue codice del sistema operativo
- **Modalità user (user mode):**
Le applicazioni vengono eseguite in modalità user, senza poter fare accesso alle risorse privilegiate hardware del calcolatore

Interrupt, errore, ecall, ebreak

Interrupt (interruzione)

- Eccezione causata da eventi esterni
 - Pressione di un tasto, movimento del mouse ecc.
- Asincrona rispetto all'esecuzione del programma
- **Gestione fra istruzioni di un altro programma**

Errore

- Eccezione causata da eventi interni
 - Condizioni eccezionali (overflow, divisione per zero) ecc.
- Sincrona rispetto all'esecuzione del programma

Environment Call (istruzione **ecall** del RISC-V)

- Eccezione (sincrona) causata da richiesta di un Servizio di Sistema
 - Stampa di un messaggio, lettura di un intero ecc.

Environment Break (istruzione **ebreak** del RISC-V)

- Eccezione (sincrona) causata da motivi diagnostici o debugging

Exception Handler

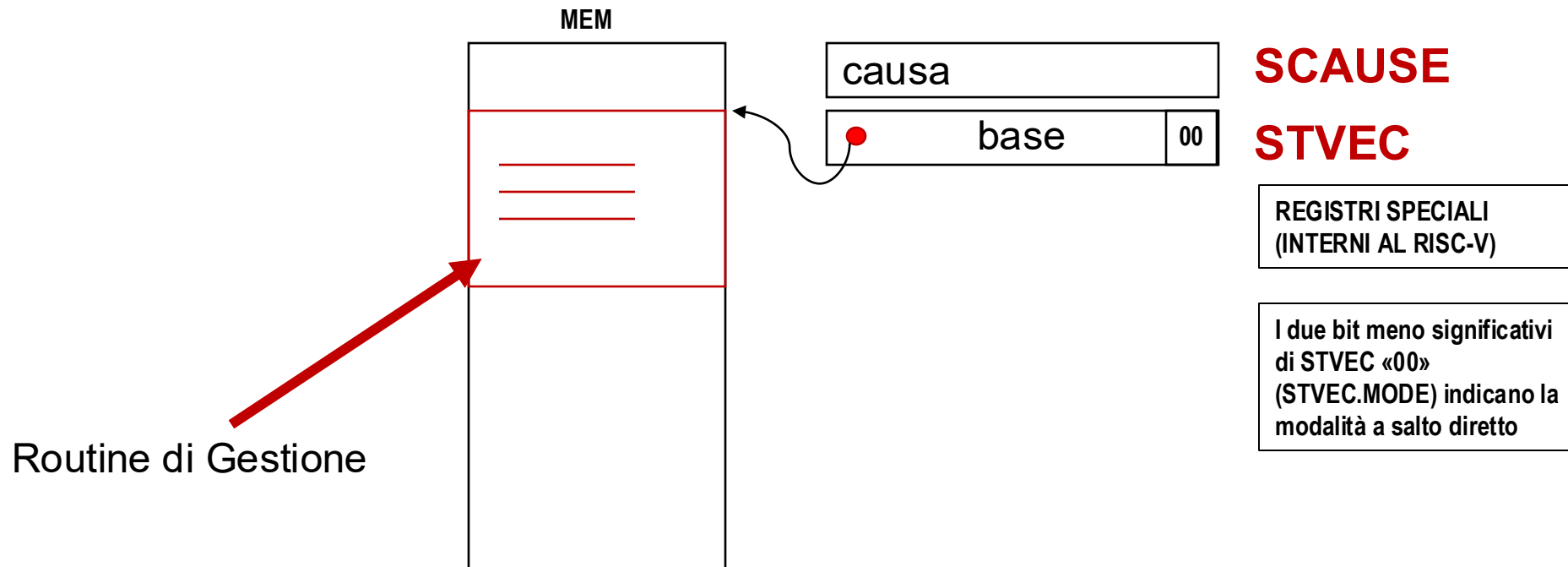
- Il **gestore dell'eccezione** deve evitare di modificare lo stato dell'applicazione
 - I contenuti di TUTTI i registri x1...x31 (e anche f0...f31) NON devono essere alterati
- Il **gestore dell'eccezione** deve essere eseguito in una modalità protetta (kernel mode)
- Le eccezioni devono essere servite una alla volta
 - e.g., disabilitando le eccezioni, se il **gestore dell'eccezione** lo ritiene opportuno può riabilitarle

Exception Handler

- Metodi di implementazione
 - **Salto diretto all'indirizzo della routine di gestione**
 - **Vettore di Interruzione**
- RISC-V: il gestore dell'eccezione salva i registri che usa (e.g., salvando i registri temporanei nello stack)
- Vengono automaticamente impostati 2 bit che indicano la modalità supervisor ed eccezioni disabilitate

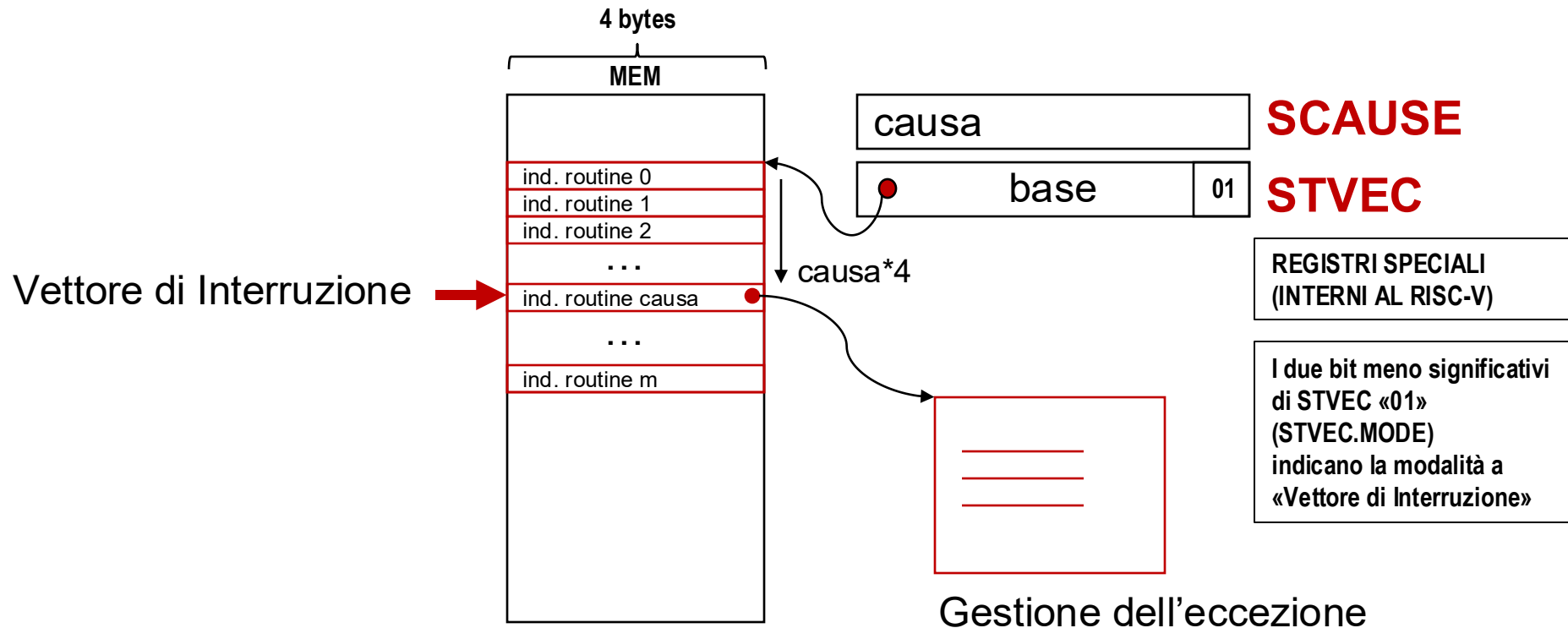
Salto diretto

- Salto diretto ad un indirizzo specifico
 - $PC \leftarrow \text{base}$
 - RISC-V: indirizzo base nel registro speciale **STVEC**
- Non è necessario prelevare l'indirizzo della routine di gestione (più veloce)
- Nella routine di gestione occorre analizzare la causa dell'eccezione



Vettore di Interruzione

- Tabella con gli indirizzi delle handler per ogni causa
 - $PC \leftarrow MEM [base + cause*4]$
 - RISC-V, ma anche 370, 68000, Vax, 80x86, ...



Salvataggio dello stato

- **Salvataggio sullo stack (Push)**
 - Vax, 68k, 80x86 (intero set dei registri)
 - RISC-V, MIPS (solo i registri necessari, potenzialmente zero)
- **Registri ausiliari (Shadow Registers)**
 - M88k, ARM
- **Salvataggio in registri speciali**
 - RISC-V, MIPS
 - Registri Speciali: EPC, CAUSE, STATUS, TVAL (o BadVaddr), ...

Eccezioni nel RISC-V

- **SEPC**
l'indirizzo dell'istruzione "colpevole"
- **SSTATUS**
i bit di abilitazione globale degli interrupt
- **SCAUSE**
codifica le possibili sorgenti di eccezione
- **STVAL** (Supervisor Trap Value)
l'indirizzo di memoria al quale si è verificato un riferimento di memoria "sbagliato" (anche chiamato "BadVAddr")
- **SIP** (Supervisor Pending Interrupts)
monitoraggio degli interrupt in attesa
- **SIE** (Supervisor Interrupt Enable)
Abilitazioni più fini per gli interrupt
- **STVEC** (Supervisor Trap Vector)
indirizzo base della lista dei «vettori di interrupt»
- **SSCRATCH**:
registro per salvataggi temporanei

Eccezioni nel RISC-V

- **STATUS** registro CSR 0x100
- **SEPC** registro CSR 0x141
- **SCAUSE** registro CSR 0x142
- **STVAL** registro CSR 0x143
- **SIP** registro CSR 0x144
- **SIE** registro CSR 0x104
- **STVEC** registro CSR 0x105
- **SSCRATCH** registro CSR 0x140

- Registri speciali: fanno parte di un banco interno di registri chiamati 'CSR' o CONTROL-STATUS REGISTERS
- Al verificarsi di un'eccezione, il controllo della CPU modifica: SEPC, SSTATUS, SCAUSE, PC

Esempio: SCAUSE Register

- Int (1 bit)
se vale 1, la sorgente è un interrupt, se vale 0 la sorgente è una eccezione
- Code (4 bits) codifica la ragione dell'eccezione
 - 0 – Instruction address misaligned
 - 2 – Illegal instruction
 - 3 – Breakpoint
 - 4 – Load address misaligned
 - 5 – Load address fault
 - 6 – Store address misaligned
 - 7 – Store address fault
 - 8 – Environment call from U-mode
 - 9 – Environment call from S-mode
 - C – Instruction page fault

...



Le istruzioni speciali

Tipo	Nome simbolico	Nome esteso
Accesso ai registri CSR	CSRRWI	Lettura/scrittura immediata CSR
	CSRRSI	Lettura/impostazione immediata CSR
	CSRRCI	Lettura/azzeramento immediato CSR
	CSRRW	Lettura/scrittura CSR
	CSRRS	Lettura/impostazione CSR
	CSRRC	Lettura/azzeramento CSR
Sistema	ECALL	Chiamata di ambiente
	EBREAK	Breakpoint di ambiente
	SRET	Ritorno da eccezione del supervisore
	WFI	Attesa di interrupt